Writing Text Data To Files In Java

Generated Aug 11, 2026 6:15 AM

What We'll Learn

By the end of this lesson, students can write and save text data to a file using Java.

Before We Start

When a Java program ends, what happens to the information entered by the user?

How can we save that information so it can be opened again later?

Let's Understand

Java can save text data using the `FileWriter` class.

First, import `FileWriter` and `IOException`.

```
import java.io.FileWriter;
import java.io.IOException;
```

Create a `FileWriter` object and provide the name of the file.

```
FileWriter writer = new FileWriter("data.txt");
```

Use the `write()` method to place text inside the file.

```
writer.write("Hello, Java!");
```

After writing the data, use the `close()` method.

```
writer.close();
```

Closing the file ensures that the data is saved correctly.

The file is usually created in the same folder where the Java program is running.

By default, `FileWriter` replaces the existing contents of a file. To add new data without removing the previous contents, place `true` after the filename.

```
FileWriter writer = new FileWriter("data.txt", true);
```

Let's Practice

Create a Java program named `SaveProduct.java`.

The program must:

1. Ask the user to enter a product name.
2. Ask the user to enter its price.
3. Ask the user to enter its quantity.
4. Save the information in a file named `product.txt`.
5. Place each piece of information on a separate line.
6. Display a confirmation message after saving the file.

The contents of the file should follow this format:

```
Product:
Price:
Quantity:
```

Let's Look at Examples

Example 1: Writing A Message To A File

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteMessage {
    public static void main(String[] args) throws IOException {
        FileWriter writer = new FileWriter("message.txt");

        writer.write("Welcome to Java programming!");

        writer.close();

        System.out.println("Message saved successfully.");
    }
}
```

This program creates a file named `message.txt` and saves a message inside it.

> **Common mistake**
>
> Students may forget to call `close()`, which can prevent the data from being saved correctly.

Example 2: Writing Multiple Lines

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteStudentData {
    public static void main(String[] args) throws IOException {
        FileWriter writer = new FileWriter("student.txt");

        writer.write("Name: Angela Cruz\n");
        writer.write("Grade Level: 11\n");
        writer.write("Section: Java");

        writer.close();

        System.out.println("Student data saved successfully.");
    }
}
```

The `\n` character moves the next text to a new line.

> **Common mistake**
>
> Students may forget to add `\n`, causing all the information to appear on one line.

Example 3: Saving User Input

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class SaveUserInput {
    public static void main(String[] args) throws IOException {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = input.nextLine();

        System.out.print("Enter your favorite subject: ");
        String subject = input.nextLine();

        FileWriter writer = new FileWriter("profile.txt");

        writer.write("Name: " + name + "\n");
        writer.write("Favorite Subject: " + subject);

        writer.close();
        input.close();

        System.out.println("Profile saved successfully.");
    }
}
```

This program collects information using `Scanner` and saves it in `profile.txt`.

> **Common mistake**
>
> Students may create the `FileWriter` but forget to use the `write()` method before closing the file.

Resources / References

- Java `FileWriter` class
- Java `IOException` class
- Java `Scanner` class